LAPORAN TUGAS INDIVIDU

PEMROGRAMAN WEB SERVICE/API

PEMBUATAN RESTFUL API CRUD BIODATA

Nama : Muhammad Fatih

NIM : 2410058

Mata Kuliah : Pemrograman Web Service/API

Disusun untuk memenuhi tugas individu mata kuliah Pemrograman Web Service/API.

1. Pendahuluan

Laporan ini menjelaskan proses pembuatan RESTful API sederhana untuk mengelola data biodata menggunakan Node.js dan Express. API menyediakan operasi Create, Read, Update, dan Delete (CRUD) dengan penyimpanan data sementara menggunakan Array di dalam aplikasi, tanpa menggunakan database.

2. Tujuan

Tujuan pembuatan program ini adalah memahami konsep RESTful API, penggunaan metode HTTP, pengolahan data JSON, pengelolaan data menggunakan Array JavaScript, serta penerapan validasi input dan penanganan kesalahan melalui HTTP status code.

3. Alat dan Teknologi

Komponen	Keterangan
Node.js	Runtime JavaScript untuk menjalankan aplikasi server.
Express	Framework Node.js untuk membuat route dan menangani request/response HT TP.
Visual Studio Code	Editor untuk menulis dan mengelola kode program.
Postman	Aplikasi untuk mengirim request dan menguji endpoint API.
JSON	Format pertukaran data pada request dan response API.

4. Struktur Data Biodata

Data biodata disimpan dalam sebuah Array bernama **biodata**. Setiap elemen Array berbentuk objek dengan atribut sebagai berikut:

Atribut	Tipe Data	Keterangan
id	Number	Identitas unik biodata yang dibuat secara berurutan.
nama	String	Nama pemilik biodata.
umur	Number	Umur pemilik biodata.
alamat	String	Alamat pemilik biodata.
pekerjaan	String	Pekerjaan pemilik biodata.

Pada saat aplikasi pertama dijalankan, terdapat dua data awal, yaitu Andi Saputra dengan ID 1 dan Siti Aminah dengan ID 2. Data yang ditambahkan melalui POST disimpan di Array selama proses server berjalan. Karena belum menggunakan database, data tambahan akan kembali ke kondisi awal ketika server dihentikan dan dijalankan ulang.

5. Implementasi Endpoint CRUD

Seluruh endpoint menggunakan awalan URL **http://localhost:3000/api/biodata**.

5.1 GET seluruh biodata

Method: **GET** | URL: **/api/biodata** | Status berhasil: **200 OK**. Endpoint ini mengirimkan seluruh data biodata yang tersimpan di Array. Response berisi status, message, dan data.

5.2 GET biodata berdasarkan ID

Method: **GET** | URL: **/api/biodata/:id** | Status berhasil: **200 OK**. Program menggunakan metode Array **find()** untuk mencari objek berdasarkan ID. Jika ID tidak ditemukan, API mengembalikan **404 Not Found**.

5.3 POST menambahkan biodata

Method: **POST** | URL: **/api/biodata** | Status berhasil: **201 Created**. Data baru dibaca dari request body JSON, dibuatkan ID melalui variabel **nextId**, kemudian ditambahkan ke Array menggunakan **push()**. Jika field wajib tidak diisi, API mengembalikan **400 Bad Request**.

5.4 PUT memperbarui biodata

Method: **PUT** | URL: **/api/biodata/:id** | Status berhasil: **200 OK**. Program menggunakan **findIndex()** untuk menemukan posisi data dalam Array. Jika ditemukan, objek pada posisi tersebut diperbarui. Jika ID tidak ditemukan, API mengembalikan **404 Not Found**. Field yang wajib diisi juga diperiksa sebelum pembaruan dilakukan.

5.5 DELETE menghapus biodata

Method: **DELETE** | URL: **/api/biodata/:id** | Status berhasil: **200 OK**. Program menggunakan **findIndex()** untuk mencari posisi data dan **splice()** untuk menghapus data dari Array. Jika ID tidak ditemukan, API mengembalikan **404 Not Found**.

6. Validasi dan Penanganan Kesalahan

Program memeriksa kelengkapan field nama, umur, alamat, dan pekerjaan pada request POST dan PUT. Jika terdapat field wajib yang tidak diisi, server mengirim response dengan status 400 dan pesan bahwa semua field wajib diisi. Untuk pencarian, pembaruan, atau penghapusan dengan ID yang tidak tersedia, server mengirim status 404. Setiap response menggunakan format JSON dengan tiga bagian, yaitu status, message, dan data.

7. Hasil Pengujian

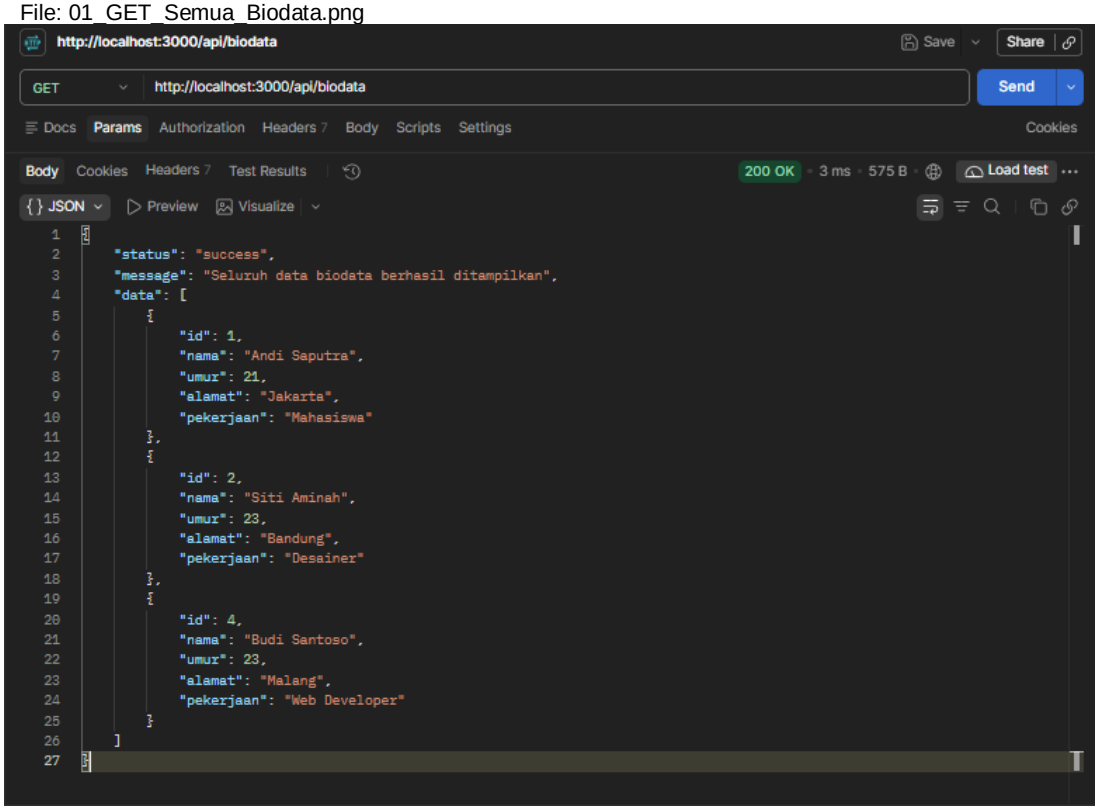
Pengujian dilakukan menggunakan Postman dengan mengirim request ke server lokal. Bukti pengujian yang dilampirkan meliputi tujuh skenario berikut:

No.	Skenario	Hasil yang diharapkan
1	GET seluruh biodata	200 OK, seluruh data ditampilkan.
2	GET biodata ID 1	200 OK, detail biodata ditemukan.
3	POST biodata lengkap	201 Created, data baru ditambahkan.
4	POST biodata tidak lengkap	400 Bad Request, data ditolak.
5	PUT biodata ID 4	200 OK, biodata berhasil diperbarui.
6	DELETE biodata ID 4	200 OK, biodata berhasil dihapus.
7	GET biodata ID 999	404 Not Found, biodata tidak ditemukan.

8. Lampiran Bukti Pengujian

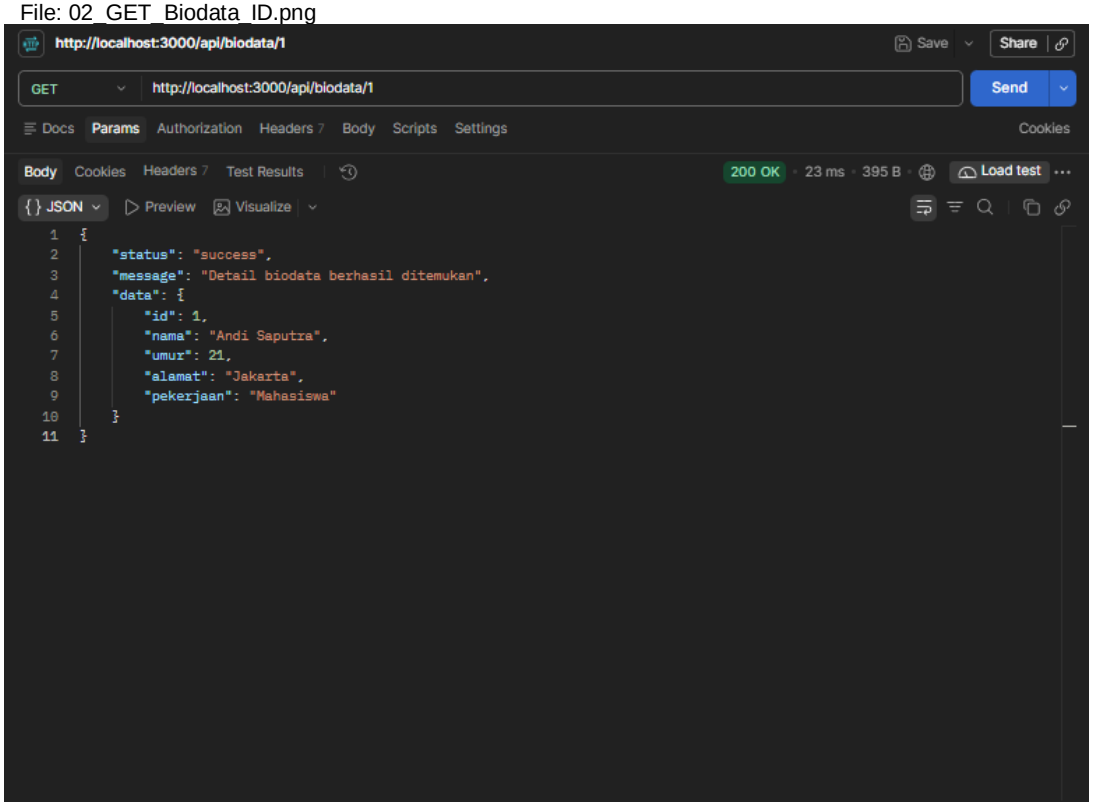
Lampiran 1. GET seluruh biodata

File: 01_GET_Semua_Biodata.png



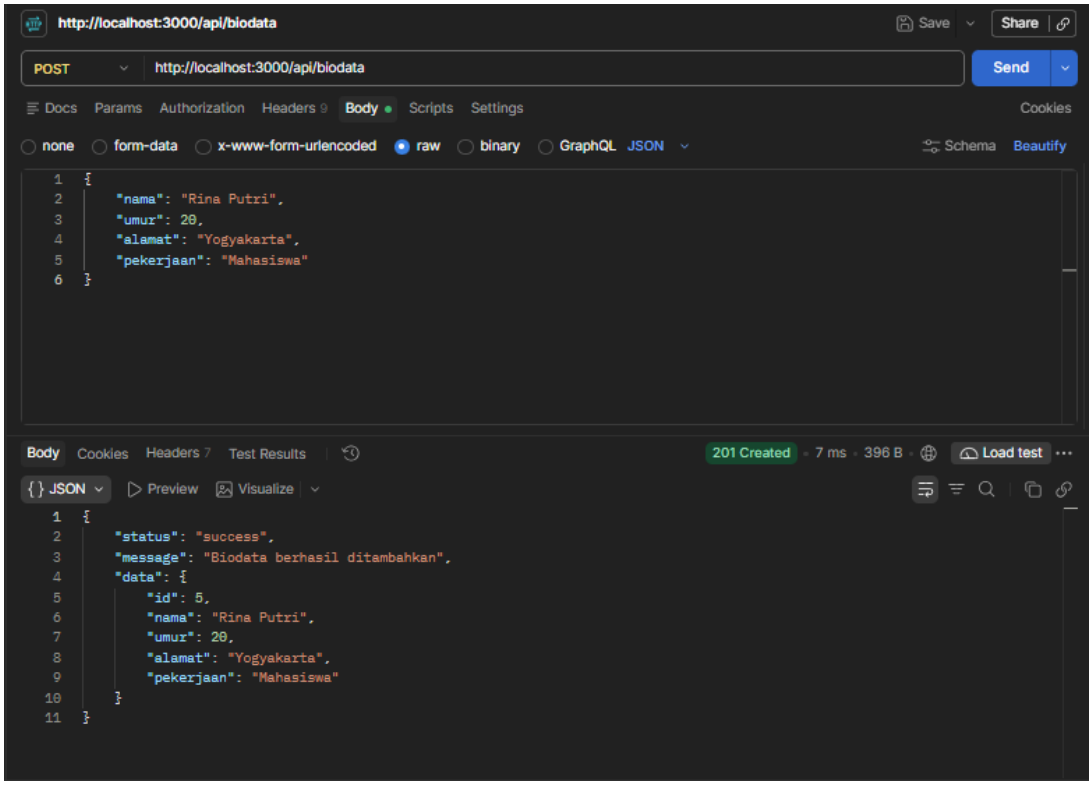
Lampiran 2. GET biodata berdasarkan ID

File: 02_GET_Biodata_ID.png



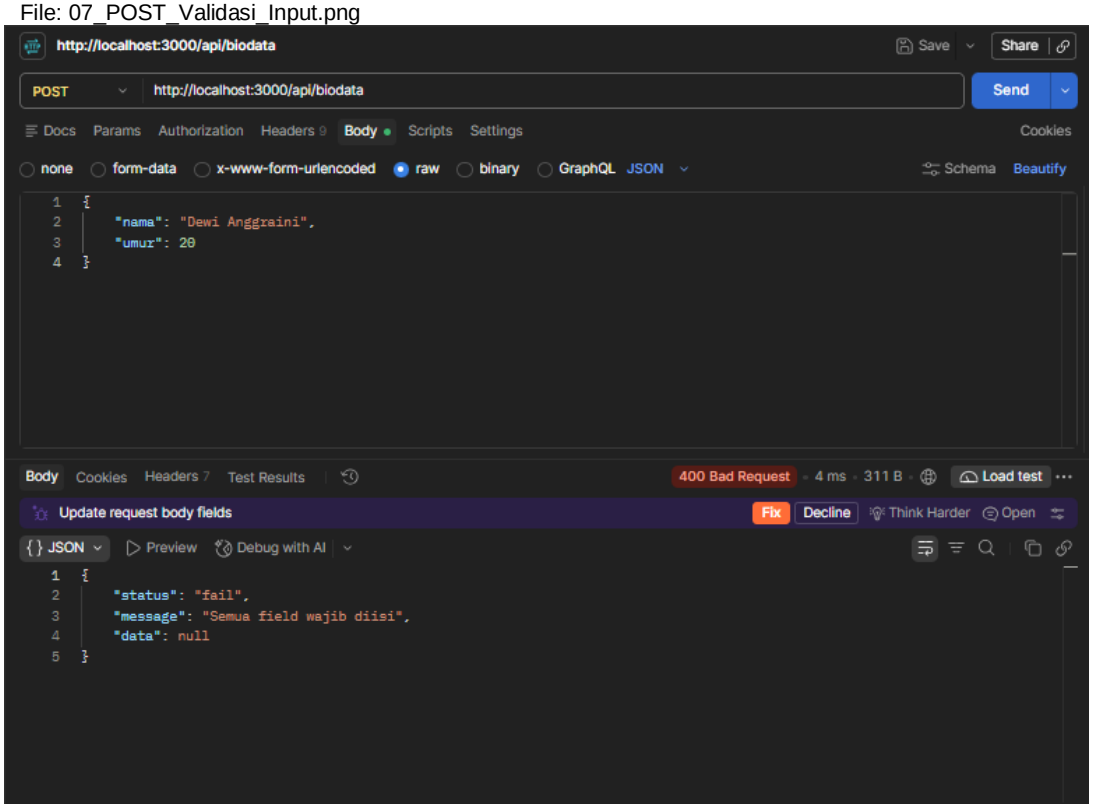
Lampiran 3. POST menambahkan biodata

File: 03_POST_Tambah_Biodata.png


http://localhost:3000/api/biodata
Save
Share
POST
http://localhost:3000/api/biodata
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"nama": "Rina Putri",
"umur": 20,
"alamat": "Yogyakarta",
"pekerjaan": "Mahasiswa"
}
Body Cookies Headers 7 Test Results
201 Created 7 ms 396 B
Load test
{} JSON
Preview
Visualize
{
"status": "success",
"message": "Biodata berhasil ditambahkan",
"data": {
"id": 5,
"nama": "Rina Putri",
"umur": 20,
"alamat": "Yogyakarta",
"pekerjaan": "Mahasiswa"
}
}

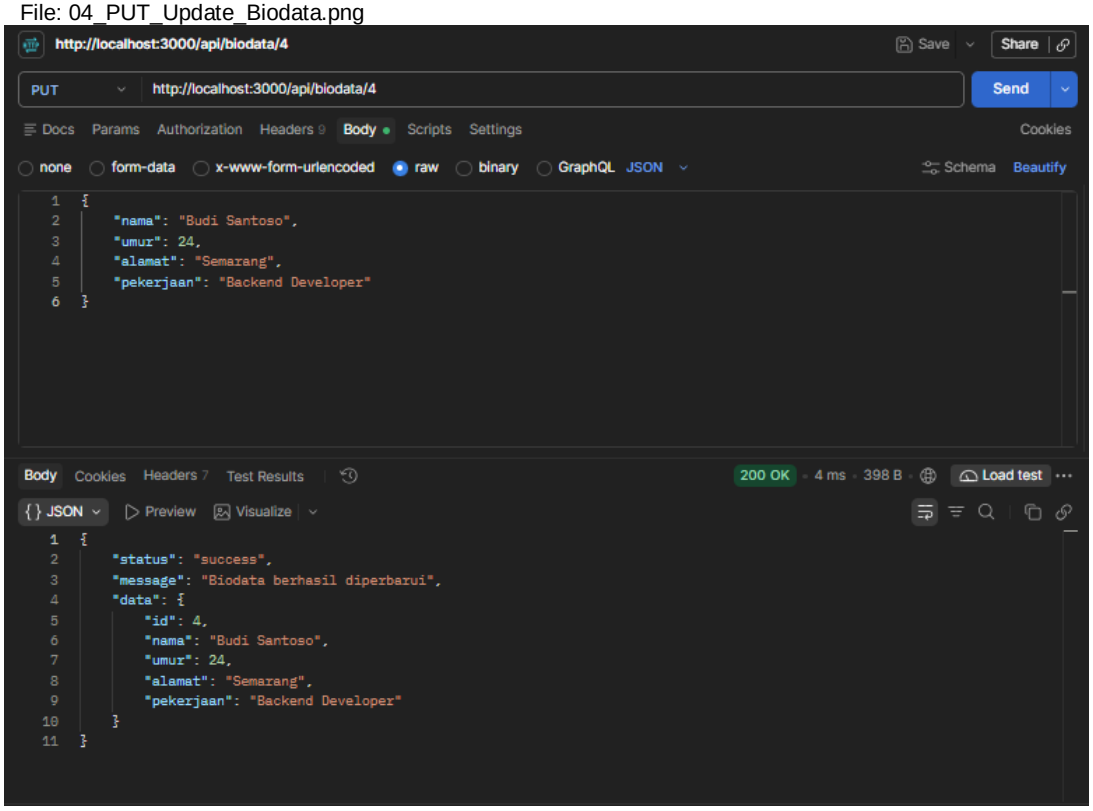
Lampiran 4. POST validasi data tidak lengkap

File: 07_POST_Validasi_Input.png



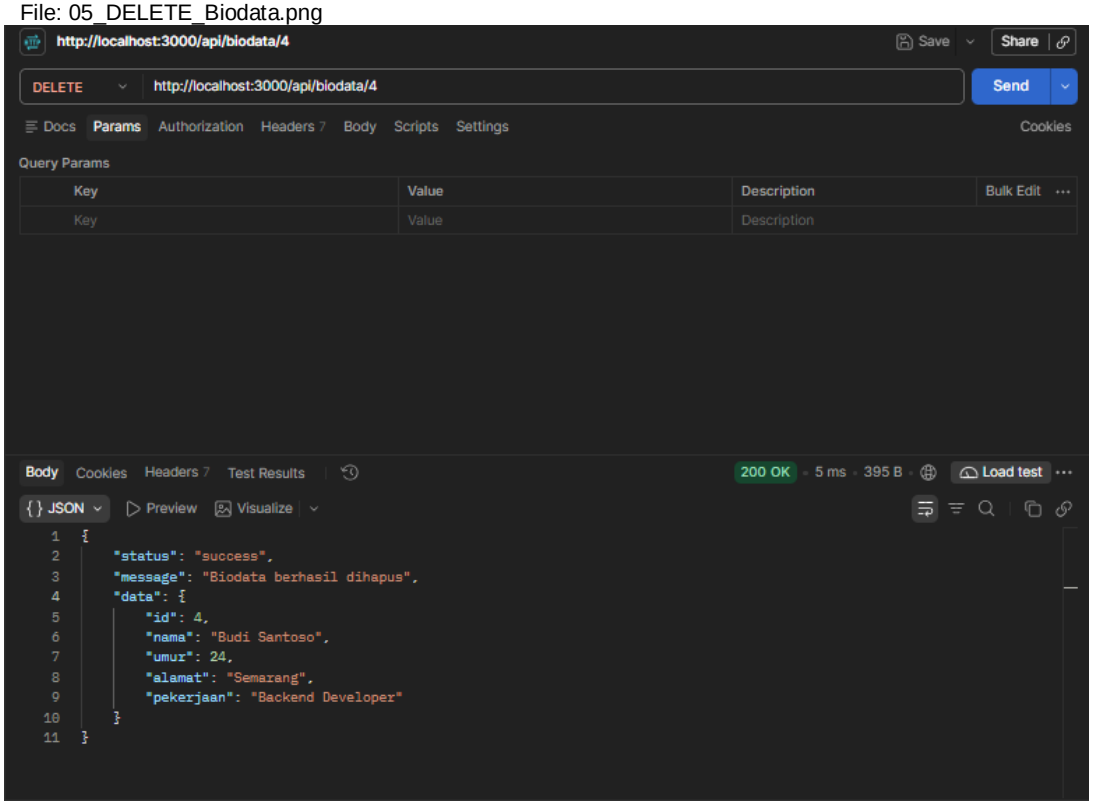
Lampiran 5. PUT memperbarui biodata ID 4

File: 04_PUT_Update_Biodata.png



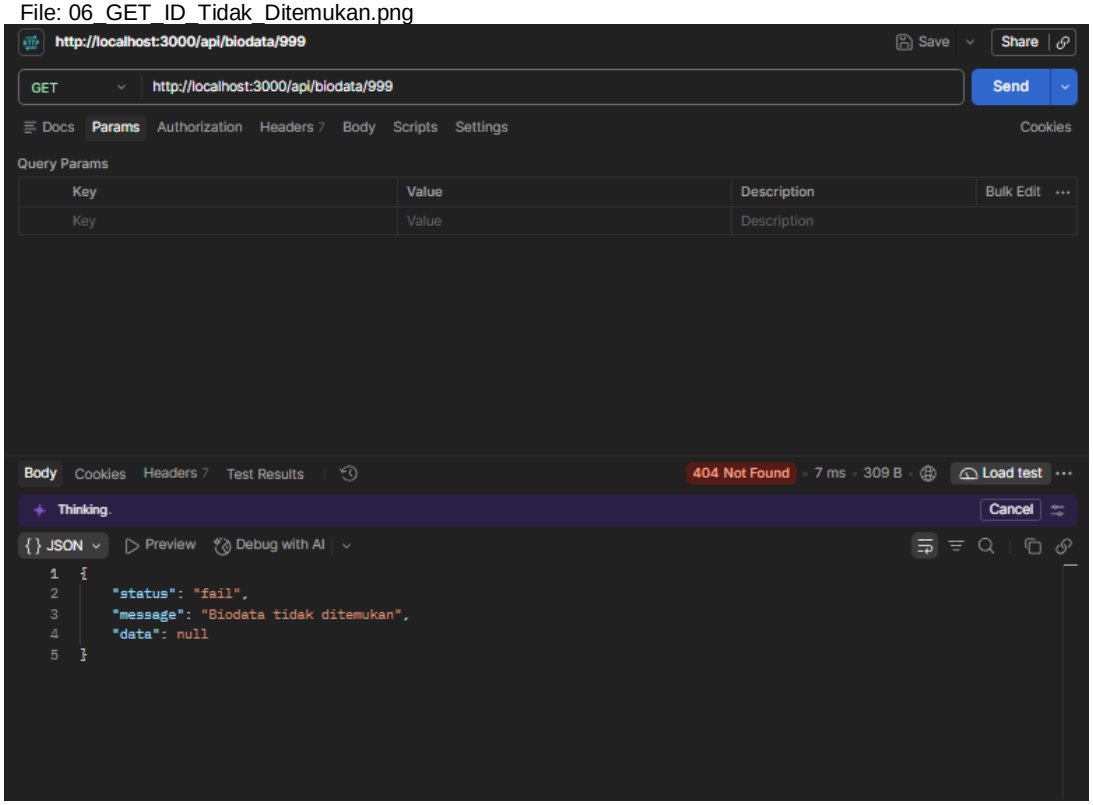
Lampiran 6. DELETE biodata ID 4

File: 05_DELETE_Biodata.png



Lampiran 7. GET ID tidak ditemukan

File: 06_GET_ID_Tidak_Ditemukan.png



9. Kesimpulan

Berdasarkan pembuatan dan pengujian yang telah dilakukan, RESTful API CRUD Biodata berhasil dibuat menggunakan Node.js dan Express dengan penyimpanan data sementara berbasis Array. API menyediakan endpoint GET, POST, PUT, dan DELETE, serta menggunakan metode Array find(), findIndex(), push(), dan splice() untuk mengelola data. Pengujian melalui Postman menunjukkan bahwa endpoint dapat memberikan response JSON dan HTTP status code sesuai kondisi, termasuk ketika data tidak ditemukan atau input tidak lengkap.

Program ini masih menggunakan penyimpanan in-memory, sehingga data yang ditambahkan atau diubah tidak bersifat permanen dan akan kembali ke data awal ketika server dimulai ulang. Pengembangan selanjutnya dapat menggunakan database agar data tersimpan secara permanen.